



**TRƯỜNG ĐẠI HỌC KINH TẾ KỸ THUẬT CÔNG NGHIỆP**  
**Khoa Công Nghệ Thông Tin**



# **LẬP TRÌNH DI ĐỘNG**

## **CHƯƠNG 6: MULTIMEDIA TRONG ANDROID**





# Chương 6 – Multimedia trong Android



## MEDIA API



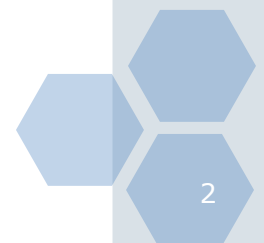
## LÀM VIỆC VỚI AUDIO



## LÀM VIỆC VỚI VIDEO



## ANIMATION TRONG ANDROID





UNETI



## 1.1 MEDIA API



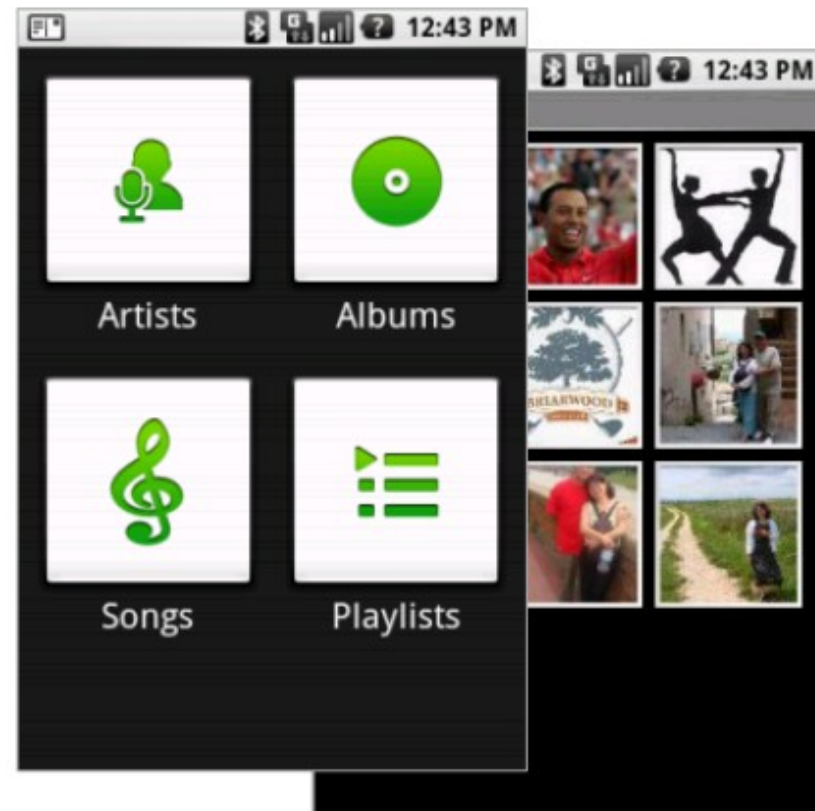
## 1.1 MEDIA API

- Android cung cấp tập hợp hỗ trợ media khá phong phú gồm cả âm thanh, hình ảnh và video
- Chia làm 2 nhóm recorder và playback
- Các lớp thư viện này đều dễ dàng sử dụng trong phát triển ứng dụng và hoàn toàn miễn phí (rất quan trọng)



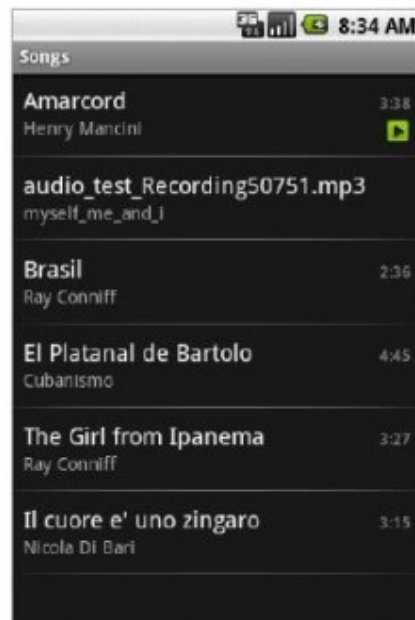
## 1.1 MEDIA API

- Các tệp tin media có thể được sử dụng – thao tác bằng những cách sau:
  - Các tệp tin media được lưu trữ bên ngay bên trong ứng dụng (các file resources)
  - Các file media độc lập có trong bộ nhớ trong của máy hoặc trong thẻ nhớ SDCARD
  - Các tài nguyên trên mạng

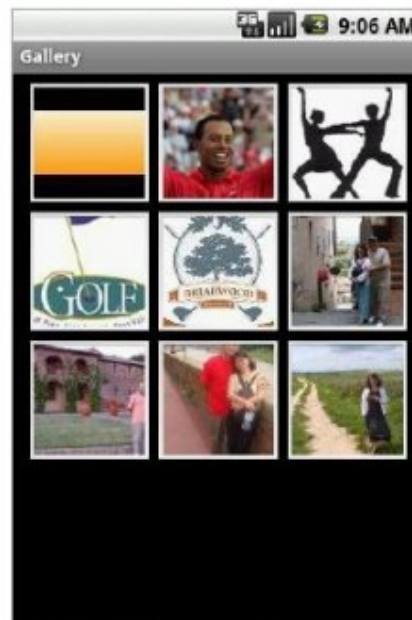


## 1.1.1 MEDIA STORE

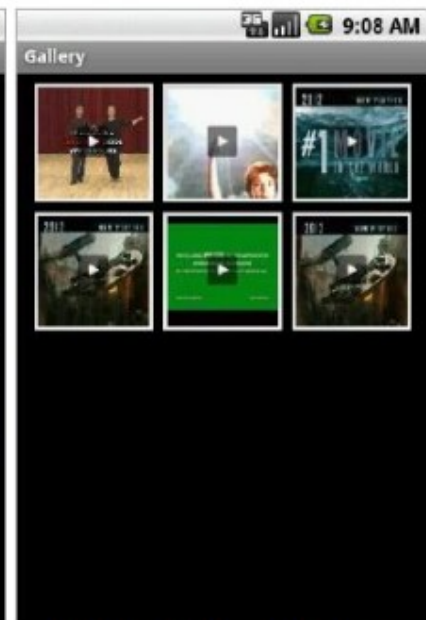
- Android OS có provider chuẩn chứa thông tin về các file media trong thiết bị
- Các ứng dụng tạo media (chụp ảnh, quay video) nên chủ động thêm các file media vào provider này



MediaStore.Audio



MediaStore.Images



MediaStore.Video



## 1.1.1 MEDIA STORE

- Có thể xem cấu trúc của MediaStore bằng cách xem file CSDL

sau: */data/data/com.android.providers.media/databases/internal.db*

// Trường hợp muốn mở các file video thì dùng Uri:

// `android.provider.MediaStore.Video.Media.EXTERNAL_CONTENT_URI`

`Uri p = android.provider.MediaStore.Images.Media.EXTERNAL_CONTENT_URI;`

`Intent myIntent = new Intent(Intent.ACTION_VIEW, p);`

`startActivity(myIntent);`

// Trường hợp muốn mở để chọn media thì sử dụng `ACTION_PICK`

`Uri x = android.provider.MediaStore.Audio.Media.EXTERNAL_CONTENT_URI;`

`Intent myIntent = new Intent(Intent.ACTION_PICK, x);`

`startActivityForResult(myIntent);`



## 1.1.2 MEDIAPLAYER & MEDIARECORDER

- Lớp MediaPlayer hỗ trợ việc playback các file audio và video
- Lớp MediaRecorder hỗ trợ việc ghi âm (ghi hình) và chuyển thành các file audio (video)
  - Chú ý: việc record phụ thuộc và việc phần cứng được hỗ trợ hay không
- Android OS chưa có cơ chế cài đặt thêm các codec mới cho audio và video, trường hợp ứng dụng muốn chạy format mới thì cần tự làm việc với i/o stream, surface view và audio stream





UNETI



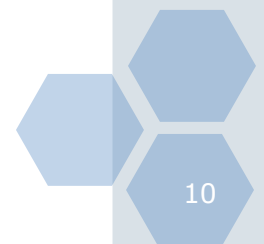
## 1.2 LÀM VIỆC VỚI AUDIO





## 1.2.1 MEDIAPLAYER

- MediaPlayer là thành phần có sẵn, nó có thể chạy các file audio và video, với file nguồn nằm trên thiết bị của bạn hoặc từ một đường dẫn URL
- MediaPlayer cung cấp các phương thức để bạn kiểm soát phát lại âm thanh (control playback of audio/video) bao gồm chạy, dừng, tua đi, tua lại,...
- MediaPlayer là một thành phần không có giao diện, nó dễ dàng giúp ta chơi một file nhạc, tuy nhiên để chơi một file video ta cần phải kết hợp nó với SurfaceView để hiển thị hình ảnh.





## 1.2.1 MEDIAPLAYER

### Public Methods

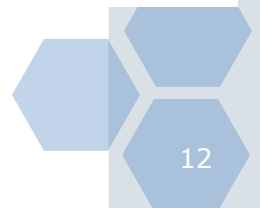
|                       |                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------|
| static<br>MediaPlayer | <code>create(Context context, Uri uri)</code><br>Convenience method to create a MediaPlayer for a given Uri.            |
| static<br>MediaPlayer | <code>create(Context context, int resid)</code><br>Convenience method to create a Media Player for a given resource id. |
| boolean               | <code>isPlaying()</code><br>Checks whether the Media Player is playing.                                                 |
| void                  | <code>pause()</code><br>Pauses play back.                                                                               |
| void                  | <code>prepare()</code><br>Prepares the player for play back ,synchronously.                                             |



## 1.2.1 MEDIAPLAYER

### Public Methods

|      |                                                                                          |
|------|------------------------------------------------------------------------------------------|
| void | <b>setLooping</b> (boolean looping)<br>Sets the player to be looping or non-looping.     |
| void | <b>setVolume</b> (float leftVolume,float rightVolume)<br>Sets the volume on this player. |
| void | <b>start</b> ()<br>Starts or resumes play back.                                          |
| void | <b>stop</b> ()<br>Stops play back after playback has been stopped or paused.             |

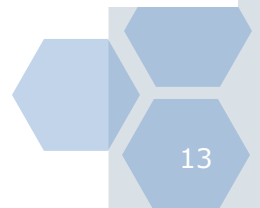




## 1.2.1 MEDIAPLAYER

### Public Methods

|      |                                                                                          |
|------|------------------------------------------------------------------------------------------|
| void | <b>setLooping</b> (boolean looping)<br>Sets the player to be looping or non-looping.     |
| void | <b>setVolume</b> (float leftVolume,float rightVolume)<br>Sets the volume on this player. |
| void | <b>start</b> ()<br>Starts or resumes play back.                                          |
| void | <b>stop</b> ()<br>Stops play back after playback has been stopped or paused.             |





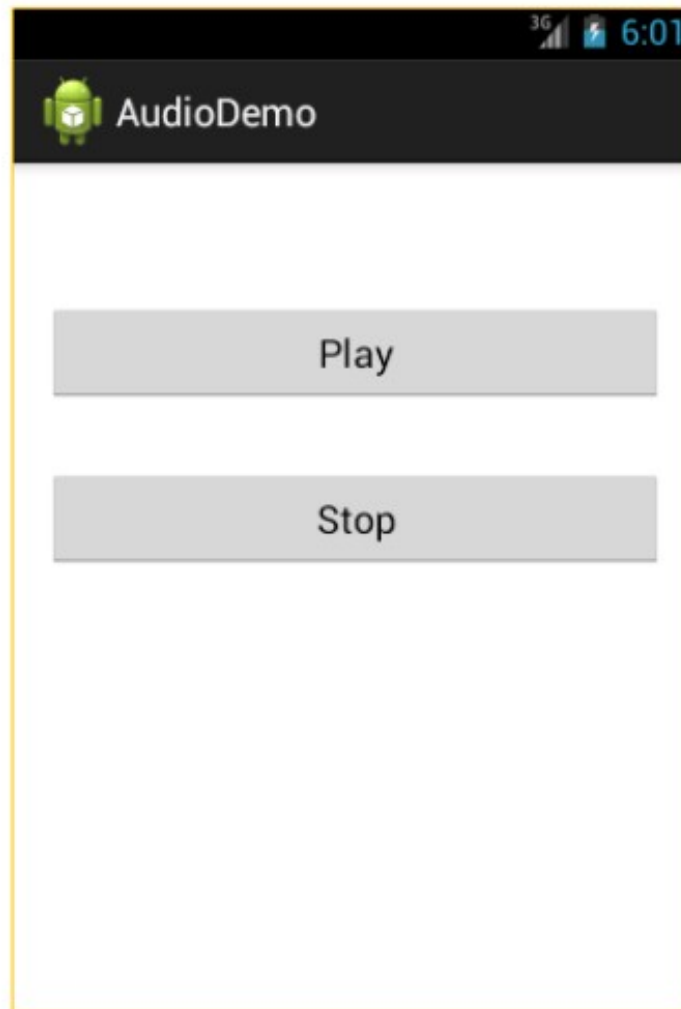
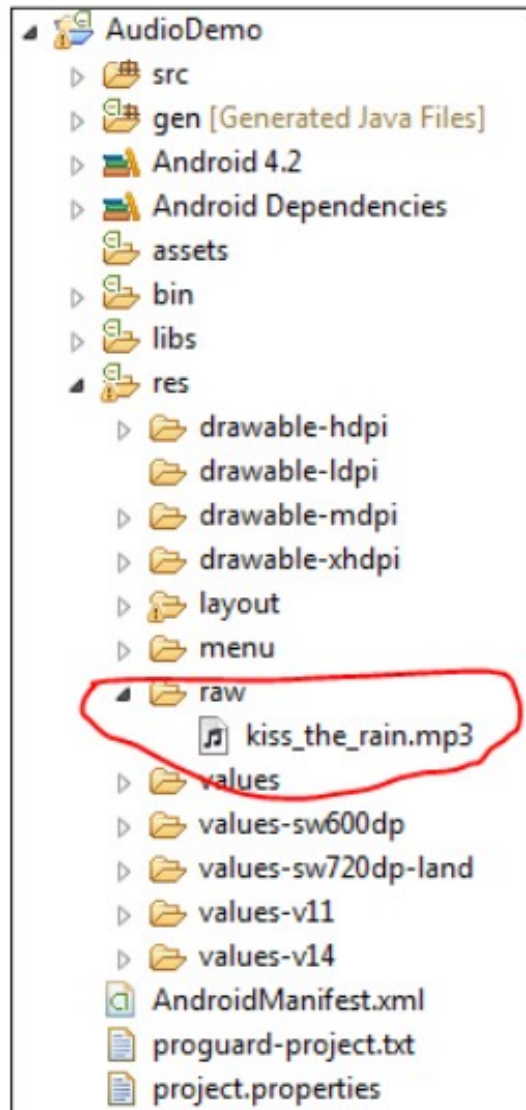
## 1.2.2 CHƠI AUDIO TỪ RESOURCE

- Tạo một file nhạc trong thư mục res/raw của dự án
- Khởi tạo đối tượng MediaPlayer thông qua hàm static create, prepare để khởi tạo các thông số cần thiết và bắt đầu chơi nhạc bằng phương thức start
- Gọi phương thức stop() để dừng lại
- Tạm dừng sử dụng phương thức pause()

Chú ý: trong thử nghiệm thì chạy file từ resource không cần gọi prepare

```
MediaPlayer mp = MediaPlayer.create(context,  
R.raw.sound_1);  
mp.prepare();  
mp.start();
```

## 1.2.2 CHƠI AUDIO TỪ RESOURCE





## 1.2.2 CHƠI AUDIO TỪ RESOURCE

```
public class MyPlayer extends Activity {  
    MediaPlayer mp = null;  
    public void onCreate(Bundle icle) {  
        super.onCreate(icle);  
        setContentView(R.layout.main);  
    }  
    // xử lý button STOP  
    public void btnStop(View v) {  
        if (null == mp) return;  
        if (mp.isPlaying()) {  
            mp.stop();  
            mp = null;  
        }  
    }  
}
```



## 1.2.2 CHƠI AUDIO TỪ RESOURCE

```
// xử lý button PLAY
public void btnPlay(View v) {
    try {
        mp = MediaPlayer.create(MyPlayer.this, R.raw.kiss_the_rain);
        mp.start();
        mp.setOnCompletionListener(new OnCompletionListener() {
            public void onCompletion(MediaPlayer arg0) {
                // xử lý khi đã chơi xong
            }
        });
    } catch (Exception e) { }
}
```



## 1.2.3 CHƠI AUDIO TỪ FILE/STREAM

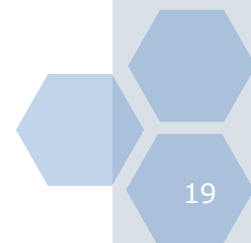
- Khởi tạo MediaPlayer thông qua hàm khởi tạo
- Gọi phương thức `setDataSource(string url)` với url là địa chỉ file hay đường dẫn trên internet
- Gọi phương thức `prepare()` để khởi tạo codec phù hợp
- Gọi phương thức `start()` để bắt đầu chơi
- Khi muốn tạm dừng hay dừng hẳn gọi các phương thức `pause()` hay `stop()`

```
String PATH_TO_FILE = "/sdcard/my_favorite_song.mp3";  
MediaPlayer mp = new MediaPlayer();  
mp.setDataSource(PATH_TO_FILE);  
mp.prepare();  
mp.start();
```



## 1.2.4 MỘT SỐ CHÚ Ý KHI PLAYBACK

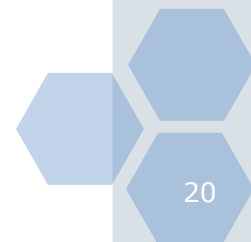
- MediaPlayer cần được **reset** hoặc **release** rồi mới được chơi lại nếu trước đó bạn **stop**
- MediaPlayer chạy ngầm, vì thế nếu bạn đóng **activity** (finish) thì audio vẫn chạy ngầm (và không có cách nào dừng nó), vì thế nên dùng `System.exit` để kết thúc ứng dụng
- Có thể chơi cùng lúc nhiều MediaPlayer và có thể thiết lập các mức volume khác nhau cũng như các nguồn ra khác nhau cho từng MediaPlayer (ví dụ chơi 2 file audio và mỗi file đổ âm thanh ra một phía của tai nghe)





## 1.2.4 MỘT SỐ CHÚ Ý KHI PLAYBACK

- Muốn ứng dụng chạy ngầm và sau khi ứng dụng quay trở lại vẫn tiếp tục điều khiển được Audio cũ, ta nên sử dụng service
- Muốn tương tác với phím điều khiển âm lượng:
  - Đăng kí broadcast receiver:  
**`android.intent.action.MEDIA_BUTTON`**
  - Điều chỉnh âm thanh bằng phương thức `setVolume(left, right)`, trong đó giá trị `left/right` là số thực nằm trong khoảng từ 0.0f đến 1.0f
- Sử dụng `AudioManager` trong trường hợp muốn tương tác nhiều hơn với phần cứng audio





UNETI



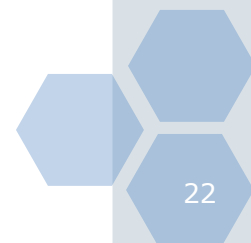
## 1.3 LÀM VIỆC VỚI VIDEO





## 1.3 LÀM VIỆC VỚI VIDEO

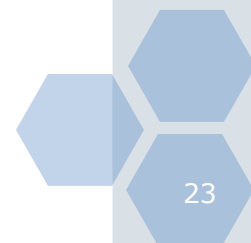
- Android OS có 2 cách để chơi lại các tập tin video:
  - Sử dụng **VideoView** kết hợp với **MediaController**
  - Sử dụng **MediaPlayer** và **SurfaceView**
- Chơi lại video không yêu cầu quyền gì đặc biệt, nhưng nếu file video ở ngoài internet, thì ứng dụng cần có quyền truy cập internet
- Phương pháp thứ 2 cho phép lập trình viên thiết lập các bộ filter cho hình ảnh phát ra thông qua hàm **setPreviewCallback(filter)**, cần có kiến thức tốt về video nếu muốn viết filter





## 1.3.1 VIDEOVIEW + MEDIACONTROLLER

- VideoView là view dùng để hiển thị dữ liệu video
- VideoView cung cấp các hàm để điều khiển quá trình chơi video:  
**start, pause, suspend, resume, stopPlayback, seekTo(millis)**
- MediaController là widget cung cấp các điều khiển cơ bản cho video, ngoài ra cũng cho lập trình viên tùy biến điều khiển các nút **next** và **prev**
- VideoView và MediaController được thiết kế để làm việc với nhau và cùng đáp ứng trải nghiệm người dùng khi chơi video (xử lý các sự kiện chạm)





## 1.3.1 VIDEOVIEW + MEDIACONTROLLER

<RelativeLayout

xmlns:android="http://schemas.android.com/apk/res/android"

xmlns:tools="http://schemas.android.com/tools"

android:layout\_width="match\_parent"

android:layout\_height="match\_parent" >

<VideoView

android:id="@+id/videoView1"

android:layout\_width="fill\_parent"

android:layout\_height="match\_parent"

android:layout\_alignParentBottom="true"

android:layout\_alignParentLeft="true"

android:layout\_alignParentRight="true"

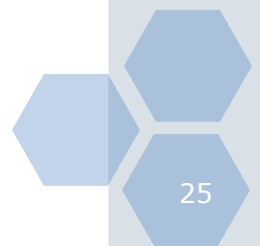
android:layout\_alignParentTop="true" />

</RelativeLayout>



## 1.3.1 VIDEOVIEW + MEDIACONTROLLER

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_main);  
    VideoView videoView = (VideoView) findViewById(R.id.videoView1);  
    videoView.setMediaController(new MediaController(this));  
    videoView.setVideoURI(Uri.parse("android.resource://" +  
        getPackageName() + "/" + R.raw.teamwork));  
    videoView.start();  
}
```





## 1.3.1 VIDEOVIEW + MEDIACONTROLLER

- MediaController mặc định là ẩn, không cần đặt lên layout khi thiết kế
- MediaController dùng hàm **setAnchorView(v)** để xác định nó sẽ được gắn vào view nào khi xuất hiện, view mặc định chính là VideoView mà nó điều khiển
- MediaController được ẩn đi sau 5s nếu không có tác động của người dùng
- VideoView mặc định giữ nguyên tỉ lệ khung hình của video mà nó chạy; muốn thiết lập tỉ lệ khác, chỉ cần điều chỉnh kích thước của

VideoView



## 1.3.2 MEDIAPLAYER + SURFACEVIEW

- MediaPlayer là bộ giải mã, luôn chạy ngầm, hỗ trợ nhiều chuẩn và giao thức video, audio, mạng
- SurfaceView là view được thiết kế với mục đích để thể hiện các hình ảnh cần có tốc độ cập nhật cao, đặc biệt thích hợp với việc thể hiện dữ liệu từ video, camera và hoạt hình
- SurfaceView có những hạn chế nhất định:
  - Thread có cơ chế cập nhật thẳng vào SurfaceView không cần qua đồng bộ
  - SurfaceView luôn chiếm một vùng màn hình và không thể bị che hoặc có bóng mờ





## 1.3.2 MEDIAPLAYER + SURFACEVIEW

```
public class MainActivity extends Activity
    implements SurfaceHolder.Callback, OnPreparedListener {

    private MediaPlayer mediaPlayer;

    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        SurfaceView vidSurface = new SurfaceView(this);
        vidSurface.getHolder().addCallback(this);
        setContentView(vidSurface);
    }
    public void surfaceChanged(SurfaceHolder s, int a, int b, int c) {
    }
    public void surfaceDestroyed(SurfaceHolder arg0) {
    }
}
```

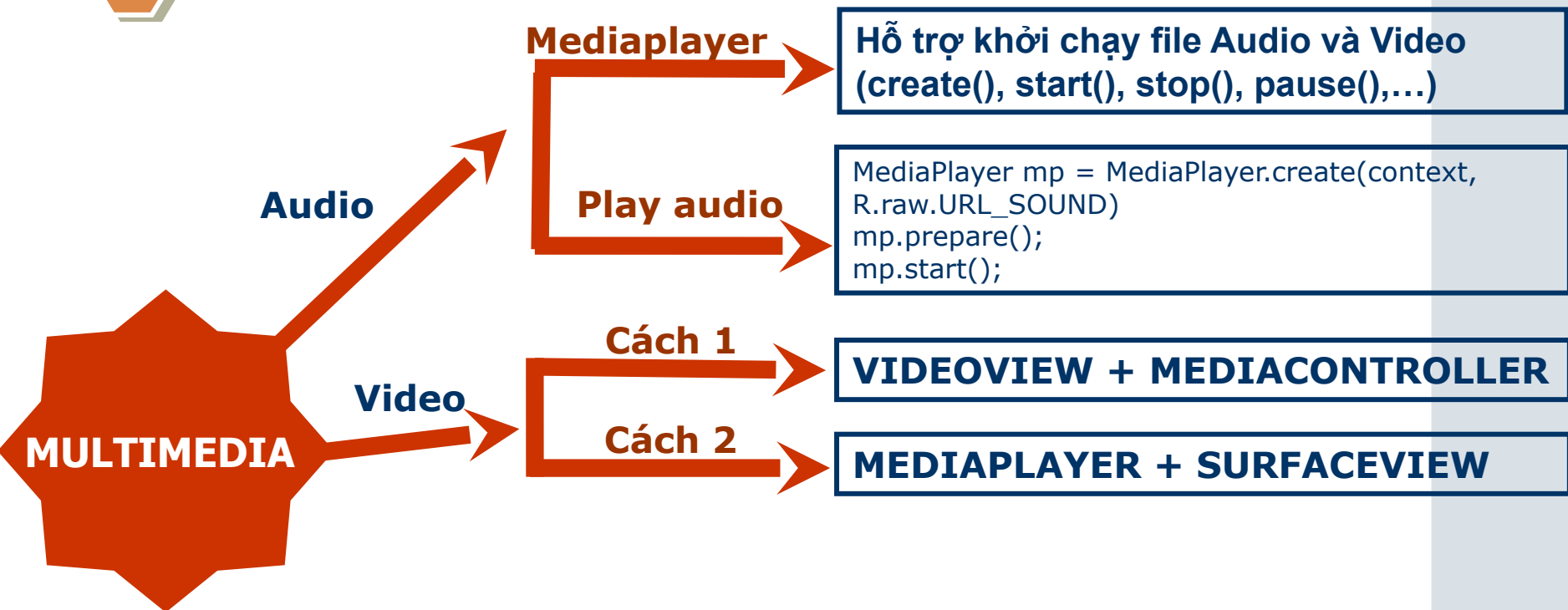


## 1.3.2 MEDIAPLAYER + SURFACEVIEW

```
public void surfaceCreated(SurfaceHolder arg0) {  
    try {  
        mediaPlayer = new MediaPlayer();  
        mediaPlayer.setDisplay(arg0);  
        mediaPlayer.setDataSource(this,  
            Uri.parse("android.resource://" +  
                getPackageName() + "/" + R.raw.teamwork));  
        mediaPlayer.prepare();  
        mediaPlayer.setOnPreparedListener(this);  
        mediaPlayer.setAudioStreamType(AudioManager.STREAM_MUSIC);  
    } catch (Exception e) { }  
}  
  
public void onPrepared(MediaPlayer mp) { mediaPlayer.start(); }  
}
```



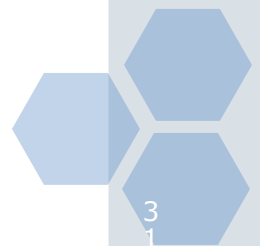
# Tổng kết





# Câu hỏi lý thuyết

1. Nêu ưu và nhược điểm của 2 cách khởi chạy video trên ứng dụng Android
2. Tại sao âm thanh có thể chạy dưới nền khi ứng dụng được ẩn khỏi màn hình chính





# Bài tập

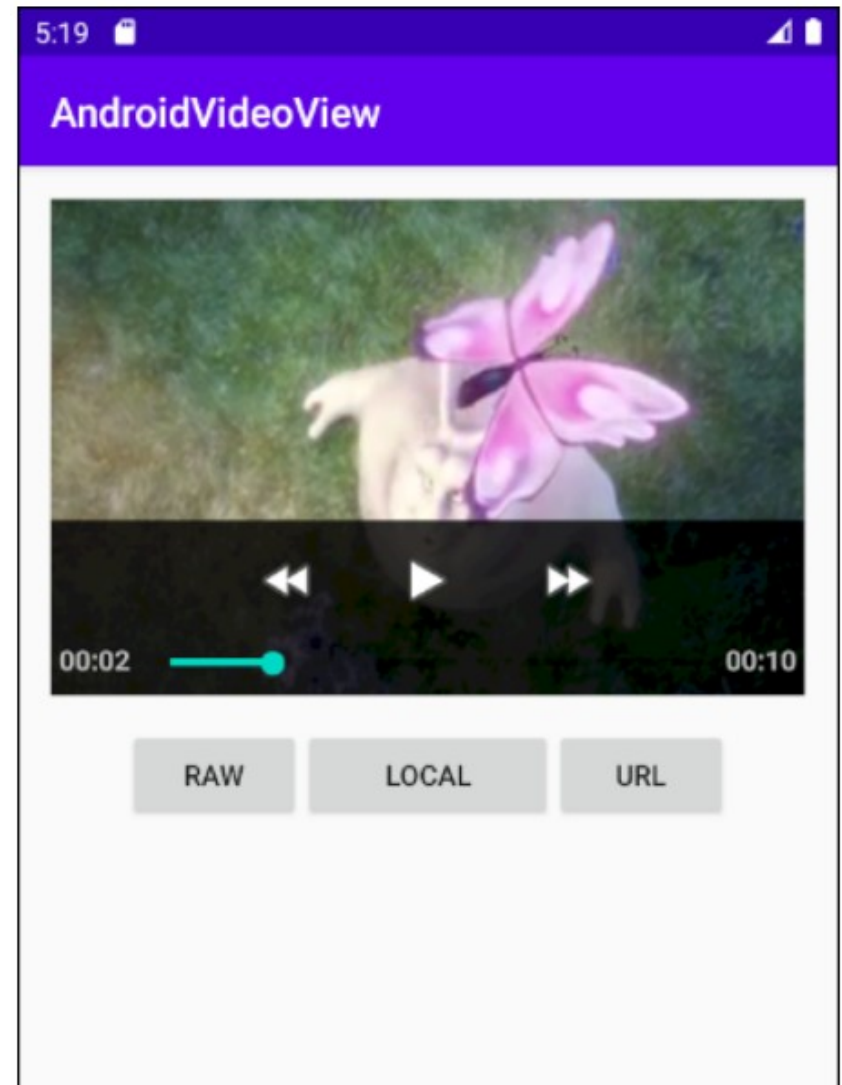
## 1. Xây dựng ứng dụng phát audio như sau





# Bài tập

## 2. Xây dựng ứng dụng phát video sau





# Bài tập

3. Xây dựng ứng dụng như sau. Chọn animation thì ImageView sẽ được áp dụng animation tương ứng

